

```
# Copyright 2021 DeepMind Technologies Limited
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
```

```
# Install dependencies for Google Colab.
# If you want to run this notebook on your own machine, you can skip this cell
!pip install dm-haiku
!pip install einops

!mkdir /content/perceiver
!touch /content/perceiver/__init__.py
!wget -O /content/perceiver/io_processors.py https://raw.githubusercontent.com/deepmind/deepmind-research/master/perceiver/io\_processors.py
!wget -O /content/perceiver/perceiver.py https://raw.githubusercontent.com/deepmind/deepmind-research/master/perceiver/perceiver.py
!wget -O /content/perceiver/position_encoding.py https://raw.githubusercontent.com/deepmind/deepmind-research/master/perceiver/
```

```
Collecting dm-haiku
  Downloading dm_haiku-0.0.16-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: absl-py>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from dm-haiku) (1.4.0)
Collecting jmp>=0.0.2 (from dm-haiku)
  Downloading jmp-0.0.4-py3-none-any.whl.metadata (8.9 kB)
Requirement already satisfied: numpy>=1.18.0 in /usr/local/lib/python3.12/dist-packages (from dm-haiku) (2.0.2)
Requirement already satisfied: tabulate>=0.8.9 in /usr/local/lib/python3.12/dist-packages (from dm-haiku) (0.9.0)
Downloading dm_haiku-0.0.16-py3-none-any.whl (374 kB)
    374.3/374.3 kB 9.1 MB/s eta 0:00:00
Downloading jmp-0.0.4-py3-none-any.whl (18 kB)
Installing collected packages: jmp, dm-haiku
Successfully installed dm-haiku-0.0.16 jmp-0.0.4
Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (0.8.2)
--2026-04-12 02:50:52-- https://raw.githubusercontent.com/deepmind/deepmind-research/master/perceiver/io\_processors.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 29409 (29K) [text/plain]
Saving to: '/content/perceiver/io_processors.py'

/content/perceiver/ 100%[=====>] 28.72K --.-KB/s in 0s

2026-04-12 02:50:53 (110 MB/s) - '/content/perceiver/io_processors.py' saved [29409/29409]

--2026-04-12 02:50:53-- https://raw.githubusercontent.com/deepmind/deepmind-research/master/perceiver/perceiver.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 30254 (30K) [text/plain]
Saving to: '/content/perceiver/perceiver.py'

/content/perceiver/ 100%[=====>] 29.54K --.-KB/s in 0.002s

2026-04-12 02:50:53 (14.1 MB/s) - '/content/perceiver/perceiver.py' saved [30254/30254]

--2026-04-12 02:50:53-- https://raw.githubusercontent.com/deepmind/deepmind-research/master/perceiver/position\_encoding.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8107 (7.9K) [text/plain]
Saving to: '/content/perceiver/position_encoding.py'

/content/perceiver/ 100%[=====>] 7.92K --.-KB/s in 0s

2026-04-12 02:50:53 (75.0 MB/s) - '/content/perceiver/position_encoding.py' saved [8107/8107]
```

> Imports

[Show code](#)

> Model construction

[Show code](#)

> Function to compute flow between pairs of images

[Show code](#)

> Load parameters from checkpoint

[Show code](#)

```
--2026-04-12 02:50:57-- https://storage.googleapis.com/perceiver\_io/optical\_flow\_checkpoint.pystate
Resolving storage.googleapis.com (storage.googleapis.com)... 74.125.200.207, 74.125.130.207, 74.125.68.207, ...
Connecting to storage.googleapis.com (storage.googleapis.com)|74.125.200.207|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 164271840 (157M) [application/octet-stream]
Saving to: 'optical_flow_checkpoint.pystate'
```

```
optical_flow_checkp 100%[=====>] 156.66M 14.7MB/s in 13s
```

```
2026-04-12 02:51:11 (11.7 MB/s) - 'optical_flow_checkpoint.pystate' saved [164271840/164271840]
```

```
# Download two example frames from the Sintel dataset.
# These files are obtained from the Sintel dataset test split,
# downloaded from http://sintel.is.tue.mpg.de/downloads.
# They correspond to MPI-Sintel-testing/test/clean/cave_3/frame_0001.png
# and MPI-Sintel-testing/test/clean/cave_3/frame_0002.png.
#
# Citation for Sintel dataset:
# D. J. Butler, J. Wulff, G. B. Stanley, and M. J. Black.
# A naturalistic open source movie for optical flow evaluation.
# European Conf. on Computer Vision (ECCV), 2012.
# https://files.is.tue.mpg.de/black/papers/ButlerECCV2012.pdf
#
# The Sintel images are originally generated for the Durian Open Movie project
# and are licensed under the Creative Commons Attribution 3.0 license (https://durian.blender.org/sharing/).
# The images are copyrighted by the Blender Foundation (https://durian.blender.org).
```

```
!wget -O sintel_frame1.png https://storage.googleapis.com/perceiver_io/sintel_frame1.png
!wget -O sintel_frame2.png https://storage.googleapis.com/perceiver_io/sintel_frame2.png
```

```
with open("sintel_frame1.png", "rb") as f:
    im1 = imageio.imread(f)
with open("sintel_frame2.png", "rb") as f:
    im2 = imageio.imread(f)
```

```
--2026-04-12 02:51:12-- https://storage.googleapis.com/perceiver\_io/sintel\_frame1.png
Resolving storage.googleapis.com (storage.googleapis.com)... 74.125.200.207, 74.125.130.207, 74.125.68.207, ...
Connecting to storage.googleapis.com (storage.googleapis.com)|74.125.200.207|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 710411 (694K) [application/octet-stream]
Saving to: 'sintel_frame1.png'
```

```
sintel_frame1.png 100%[=====>] 693.76K 578KB/s in 1.2s
```

```
2026-04-12 02:51:13 (578 KB/s) - 'sintel_frame1.png' saved [710411/710411]
```

```
--2026-04-12 02:51:14-- https://storage.googleapis.com/perceiver\_io/sintel\_frame2.png
Resolving storage.googleapis.com (storage.googleapis.com)... 74.125.200.207, 74.125.130.207, 74.125.68.207, ...
Connecting to storage.googleapis.com (storage.googleapis.com)|74.125.200.207|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 712519 (696K) [application/octet-stream]
Saving to: 'sintel_frame2.png'
```

```
sintel_frame2.png 100%[=====>] 695.82K 577KB/s in 1.2s
```

```
2026-04-12 02:51:15 (577 KB/s) - 'sintel_frame2.png' saved [712519/712519]
```

```
/tmp/ipykernel_4343/3776874795.py:22: DeprecationWarning: Starting with ImageIO v3 the behavior of this function will switch to
    im1 = imageio.imread(f)
/tmp/ipykernel_4343/3776874795.py:24: DeprecationWarning: Starting with ImageIO v3 the behavior of this function will switch to
    im2 = imageio.imread(f)
```

✓ Image Utility Functions

```
##@title Image Utility Functions
```

```
def normalize(im):  
    return im / 255.0 * 2 - 1  
  
def visualize_flow(flow):  
    flow = np.array(flow)  
    # Use Hue, Saturation, Value colour model  
    hsv = np.zeros((flow.shape[0], flow.shape[1], 3), dtype=np.uint8)  
    hsv[..., 2] = 255  
  
    mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])  
    hsv[..., 0] = ang / np.pi / 2 * 180  
    hsv[..., 1] = np.clip(mag * 255 / 24, 0, 255)  
    bgr = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)  
    plt.imshow(bgr)
```

```
# Compute optical flow
```

```
# Divide images into patches, compute flow between corresponding patches  
# of both images, and stitch the flows together  
grid_indices = compute_grid_indices(im1.shape)  
flow = compute_optical_flow(params, rng, normalize(im1), normalize(im2), grid_indices)
```

```
# Visualize the computed flow  
visualize_flow(flow[0])
```